UNIVERSIDAD DON BOSCO

Facultad De Ingeniería

Escuela De Computación

Técnico en Ingeniería en Ciencias de la Computación

“Educational Control – Proyecto – Fase 1”

Diseño y Programación de Software Multiplataforma DPS441 G01T (Antiguo Cuscatlán)

Integrantes:

Gabriela Stephani Figueroa Calderón FC250109 ----Coordinadora

Daniela Nicole García Guillén GG251303

Katherine Valeria Renderos Romero RR250127

Rosa Lucia Villalobos Flores VF250110

Julián Andrés Santacruz Palomino SP240085

Ciclo: 03

Docente: Emerson Francisco Cartagena Candelario

Grupo: 01T

Fecha de realización: 17 / 03 / 2026

Índice:

1. Introducción 4
2. Metodología de trabajo del equipo 5
   2.1 Enfoque de trabajo: Scrum 5
   2.2 Definición de roles del equipo 5
   2.3 Uso de Trello para la gestión del proyecto 7
      Link de Trello: 8
3.Diseño UX/UI y línea gráfica 9
   3.1 Línea grafica del sistema 11
   3.2 Paleta de colores 11
   3.3Tipografia 12
   3.4 Iconografía 12
   3.5 Uso de imágenes 13
   3.6 Diseño de tarjetas cards 14
   3.7 Navegación de la App 15
   3.8 Manejo de Estados y Retroalimentación Visual 16
   6.5 Enfoque Centrado en el Usuario 16
4.Explicación de la lógica de solución 19
   4.1 Pasos de la Lógica de Solución 19
   4.2 Resumen de los pasos: 22
4.3 Diagramas 23
   4.3.1 Diagrama de Caso de Uso 23
   4.3.2 Diagrama de Clases 24
   4.3.3 Diagrama de Entidad Relación 25
   4.3.4 Diagrama de Secuencia 26
5.Arquitectura de la aplicación 31
   Diagrama gráfico de la arquitectura de software. 31
   Backend y lógica de negocio 32
   Base de datos y modelo de datos. 32
      Modelo de Datos 33
      Base de datos 34

Tablas de la base de datos: 36
Relaciones entre tablas 41
API y protocolos de comunicación. 43
Gestión de sesiones 44
Estrategia de seguridad 44
6. Herramientas y tecnologías a utilizar 45
6.1. Lenguajes y frameworks 45
6.2. Gestor de base de datos 45
6.3. Control de versiones Git/GitHub 46
6.4. Estrategia de CI/CD y herramientas seleccionadas 46
6.5. Herramientas complementarias 46
7. Licenciamiento del proyecto 47
7.1 Tipo de Licencia Creative Commons seleccionada 47
7.2 Justificación de su elección y forma de implementación. 47
8. Presupuesto del proyecto 49
9. Cronograma de trabajo 50
Enlaces 52
Bibliografias 52

1. Introducción

El presente proyecto tiene como finalidad desarrollar **Educational Control**, una aplicación orientada a facilitar la gestión académica en instituciones educativas. Actualmente, muchas escuelas continúan utilizando métodos manuales para el registro de estudiantes, control de asistencia y gestión de calificaciones, lo que puede generar errores administrativos, pérdida de información y dificultades en el acceso oportuno a los datos académicos.

Ante esta situación, se propone el desarrollo de una aplicación multiplataforma que permita digitalizar y automatizar estos procesos mediante el uso de tecnologías modernas de desarrollo de software. La aplicación permitirá registrar estudiantes, gestionar asistencia, administrar calificaciones y generar reportes académicos, contribuyendo a mejorar la organización y eficiencia en la gestión educativa.

Además, el proyecto integra diversas prácticas y herramientas utilizadas en el desarrollo de software moderno, como metodologías ágiles de trabajo, control de versiones, diseño de interfaces de usuario y modelado de sistemas. Estas prácticas permiten estructurar el desarrollo del proyecto de manera colaborativa, organizada y orientada a resultados.

De esta manera, el proyecto **Educational Control** busca ofrecer una solución tecnológica que contribuya a optimizar los procesos administrativos de las instituciones educativas, mejorando la gestión de la información académica y facilitando el trabajo de docentes y administradores.

2. Metodología de trabajo del equipo

2.1 Enfoque de trabajo: Scrum

Para el desarrollo del proyecto se utilizará la metodología ágil **Scrum**, la cual permite organizar el trabajo del equipo mediante iteraciones cortas denominadas *Sprint*, facilitando la planificación, seguimiento y adaptación continua del desarrollo del sistema.

Scrum promueve la colaboración entre los integrantes del equipo y permite dividir el desarrollo del proyecto en pequeñas tareas que pueden ser gestionadas y supervisadas de manera constante. Esto facilita la identificación de problemas, la mejora continua del proceso de desarrollo y el cumplimiento de los objetivos del proyecto.

Mediante esta metodología, el equipo organiza las actividades del proyecto en un **Product Backlog**, donde se registran todas las funcionalidades que debe tener la aplicación para trabajar en Fase II. Posteriormente, estas tareas se priorizan y se seleccionan para desarrollarse durante cada sprint.

2.2 Definición de roles del equipo

Para una adecuada organización del trabajo, los integrantes del equipo asumen diferentes roles dentro de la metodología Scrum:

Coordinador / Scrum Master (Gabriela Stephani Figueroa Calderón)

Responsable de organizar el trabajo del equipo, coordinar las actividades del proyecto y asegurar que se cumpla la planificación establecida durante el desarrollo.

Desarrolladores

Encargados de implementar las funcionalidades del sistema, programar los diferentes módulos de la aplicación y colaborar en la solución de problemas técnicos.

Katherine Valeria Renderos Romero - RR250127

Rosa Lucía Villalobos Flores - VF250110

Julián Andrés Santacruz Palomino - SP240085

Administrador de Base de Datos (DBA)

Responsable de diseñar, estructurar y administrar la base de datos del sistema, definiendo las tablas, relaciones y estructuras necesarias para almacenar de manera segura la información académica de los estudiantes.

Daniela Nicole García Guillén - GG251303

Rosa Lucía Villalobos Flores - VF250110

Diseñador UX/UI

Responsable de elaborar los diseños de la interfaz de usuario de la aplicación, incluyendo los prototipos o Mock Ups que permitirán visualizar la apariencia y funcionamiento del sistema.

Gabriela Stephani Figueroa Calderón - FC250109

2.3 Uso de Trello para la gestión del proyecto

Para la gestión y seguimiento de las actividades del proyecto se utiliza la herramienta Trello, la cual permite organizar las tareas mediante un tablero colaborativo accesible para todos los integrantes del equipo. En el tablero se organizan las tareas en diferentes columnas que representan el flujo de trabajo del proyecto:

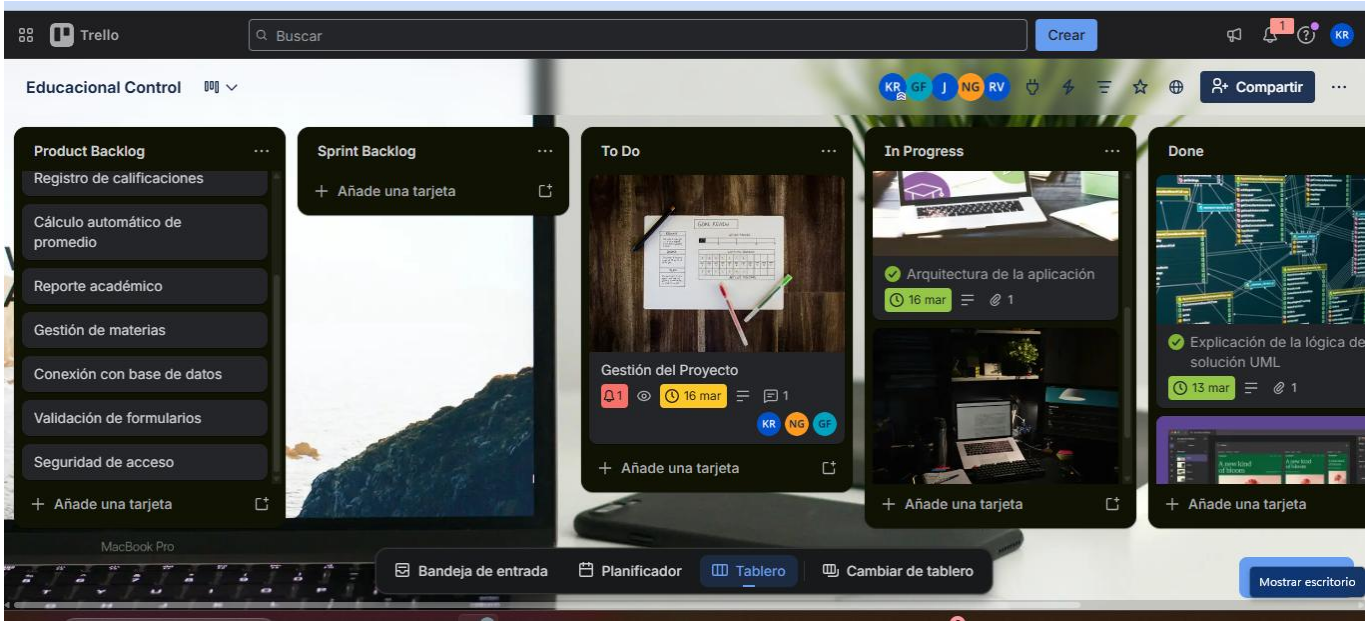
- **Product Backlog:** contiene todas las funcionalidades y requerimientos que debe tener la aplicación para realizar en la Fase 2.
- **Sprint Backlog:** incluye las tareas seleccionadas para desarrollarse durante el sprint actual en este caso la Fase 1.
- **To Do:** tareas pendientes de iniciar.
- **In Progress:** tareas que se encuentran en desarrollo.
- **Done:** tareas finalizadas.

La planificación de actividades se realiza mediante la creación de tarjetas dentro del tablero de Trello, donde se asignan tareas específicas a cada integrante del equipo. Cada tarjeta incluye una descripción de la actividad, responsable, fechas estimadas y estado de avance.

El seguimiento del proyecto se realiza moviendo las tareas entre las columnas To Do, In Progress y Done, permitiendo visualizar fácilmente el progreso del desarrollo y el cumplimiento de los objetivos de cada sprint.

Link de Trello:

https://trello.com/invite/b/69b2496efa5541d8a395ca18/ATTIc5e0ee4efd210833f315e28e39651bdb60384A71/educacional-control



3.Diseño UX/UI y línea gráfica

El diseño y la experiencia del usuario (UX) y la interfaz (UI) representa una etapa fundamental en el desarrollo de la aplicación, ya que no solo determina la apariencia visual de la aplicación, sino también la forma en que los usuarios interactúan con ella. Se centra en la experiencia global del usuario, asegurando que la aplicación sea intuitiva, eficiente y fácil de utilizar.

El diseño de **educational control** fue elaborado utilizando una herramienta llamada **Visily,** que nos permite la creación de mockups interactivos y eficientes que representan el comportamiento del sistema, y como el usuario podrá interactuar en él.

El objetivo del diseño Ux/UI del sistema educational control es:

- Garantizar una navegación intuitiva
- Mantener coherencia visual en todas las pantallas
- Mejorar la percepción de calidad de la App
- Mockups de la aplicación

Se desarrollaron Mockups que representan las principales pantallas de sistema. Estos diseños me permiten anticipar la estructura, distribución de componentes y flujos de navegación.

Las pantallas diseñadas incluyen

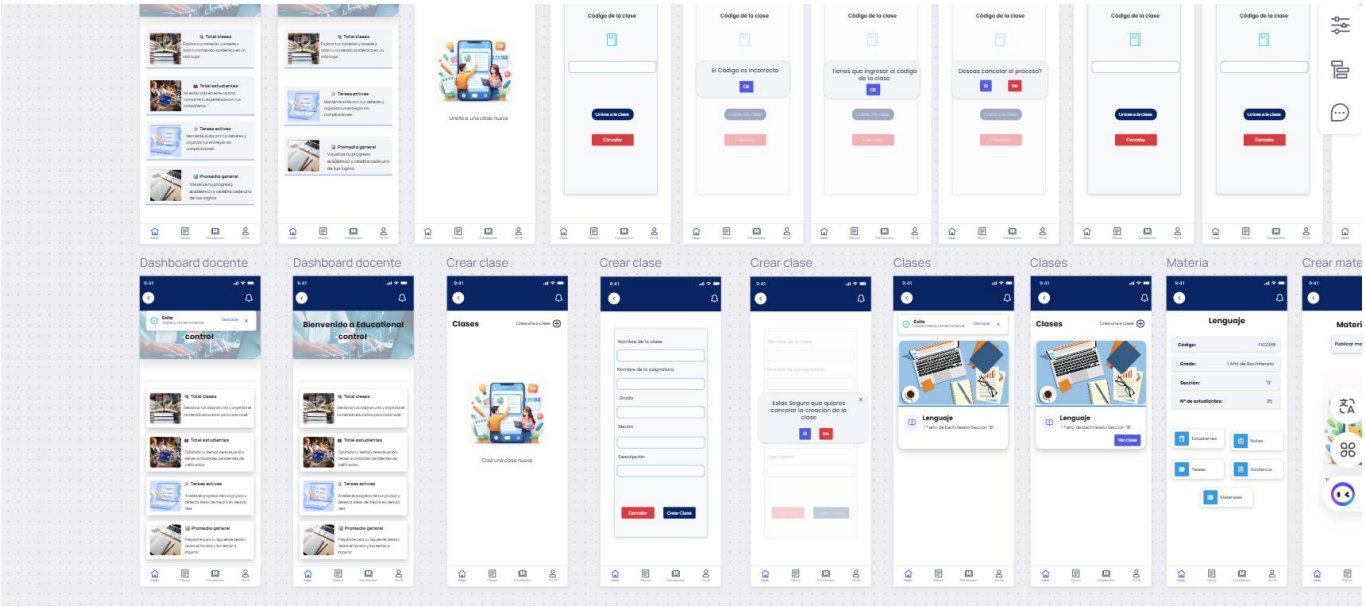
- Pantalla principal

- Pantalla de Inicio de sesión y Registrar cuenta
- Dashboard por cada perfil (Docente, Estudiante, Administrador)
- Perfil Docente
    - Dashboard Docente
    - Crear clase
    - Ver clase
    - Ver estudiantes
    - Notas
    - Tomar Asistencia
    - Publicar tarea
    - Reporte
- Perfil Estudiantes
    - Dashboard estudiante
    - Unirse a clase
    - Ver notas
    - Ver asistencia
    - Ver tareas
    - Perfil
- Perfil administrador
    - Dashboard administrador
    - Gestionar usuarios
    - Gestionar clases
    - Reportes generales

- Pantallas de error
- Pantallas de estados vacíos
- Pantalla de desconexión

3.1 Línea grafica del sistema

La línea grafica del sistema define la identidad visual del sistema y establece normas claras para garantizar uniformidad en todos los elementos de diseño. Su finalidad es lograr coherencia visual, profesionalismo y consistencia en cada pantalla de la aplicación



3.2 Paleta de colores

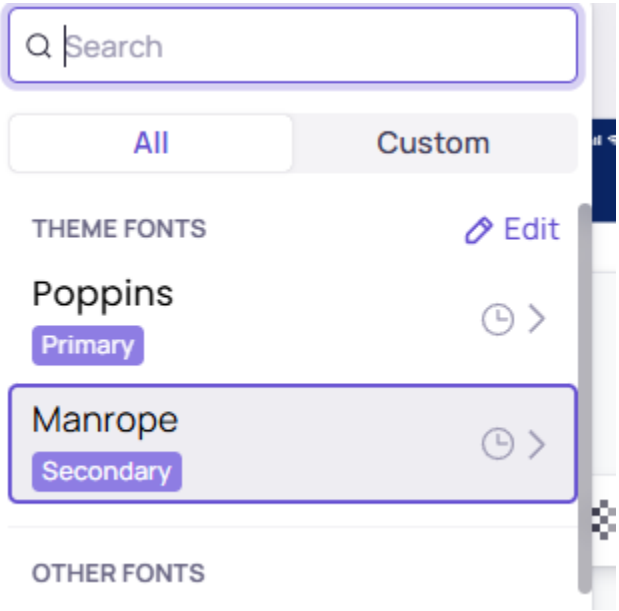
- **Color primario:** #0a2564
- **Color secundario:** #f8f9fa
- **Color de fondo:** #ffffff
- **Color de error:** #de3b40

- **Texto principal:** #171a1f

3.3Tipografia

Busca garantizar legibilidad y jerarquía visual. Se establecen distintos tamaños y estilos para diferenciar

- **Títulos principales:** Poppins (Bold)
- **Subtítulos:** Poppins (Semi Bold)
- **Texto general:** Manrope
- **Botones y etiquetas :** Manrope



3.4 Iconografía

La interfaz incorpora iconos simples y minimalistas, ubicados principalmente en:

- Barra de navegación
- Botones de acción

- Tarjetas informativas

Se mantienen siempre coherencia en:

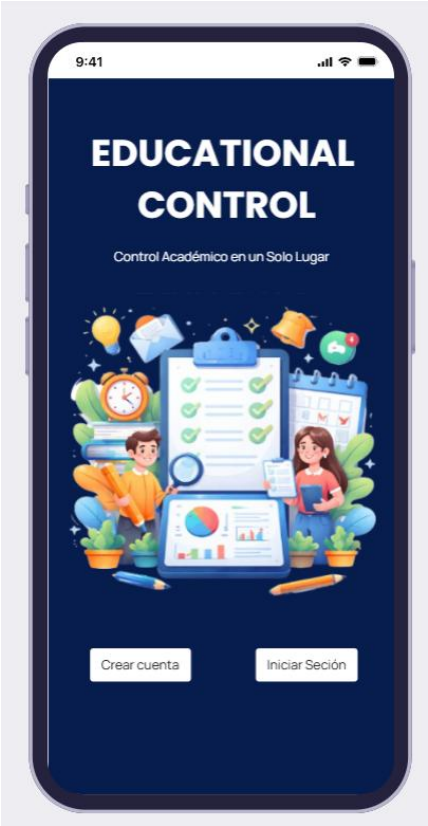
- Tamaño
- Estilo
- Colores
- Ubicación

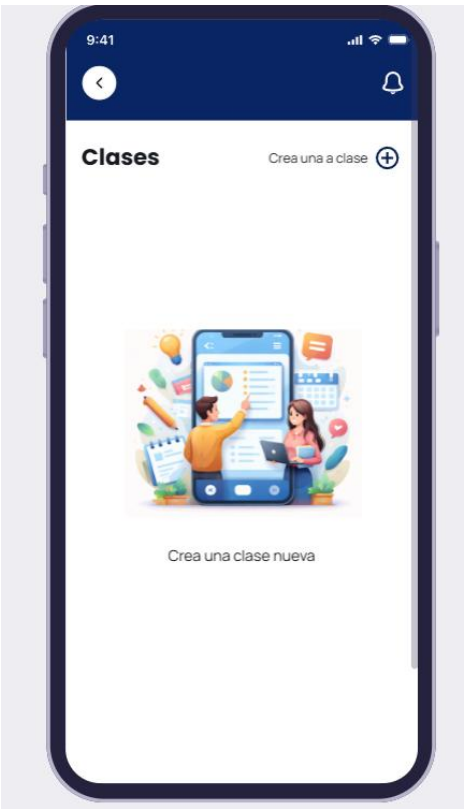
3.5 Uso de imágenes

Las imágenes utilizadas cumplen una función ilustrativa y comunicativa. Se prioriza el uso de imágenes optimizadas para garantizar un buen rendimiento.

Características considerables:

- Resolución adaptable
- Estilo moderno
- Relación con el contexto de la aplicación



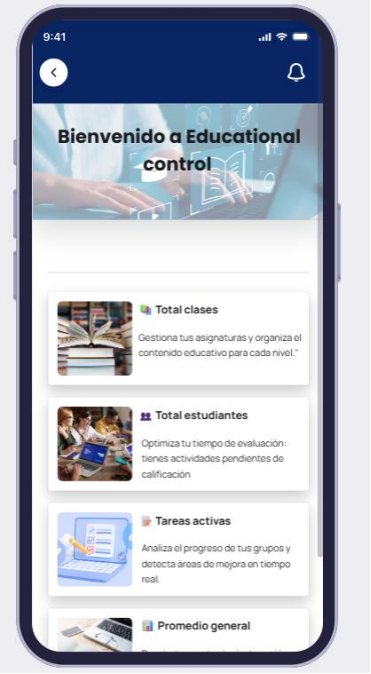


3.6 Diseño de tarjetas cards

En el dashboard se implementa el uso de tarjetas informativas para mostrar

- Total, de clases
- Total, de estudiantes
- Tareas activas
- Promedio general
- Materias registradas

El diseño tipo “card” mejora la organización de la información y permite agrupar datos relevantes de forma clara y estructurada.



3.7 Navegación de la App

El sistema implementa una navegación estructurada basada en:

- Barra superior

  Contiene el título de la pantalla y elementos de notificación, manteniendo consistencia visual.

- Navegación inferior (Bottom Navigation)

  Permite acceso rápido a:

  - Inicio
  - Clases
  - Materiales

- Perfil

Este tipo de navegación es ideal para aplicaciones móviles, ya que facilita el acceso rápido con el pulgar.

3.8 Manejo de Estados y Retroalimentación Visual

Uno de los aspectos importantes del diseño es el manejo de estados especiales, los cuales mejoran la experiencia del usuario.

Se diseñaron:

- Mensajes de éxito (Ej. clase creada correctamente).
- Confirmaciones antes de cancelar una acción.
- Validación de código incorrecto.
- Estado sin datos.
- Pantallas con botones deshabilitados cuando no se cumplen condiciones.
- Esto evita confusión y reduce errores por parte del usuario.

6.5 Enfoque Centrado en el Usuario

El diseño de la app fue desarrollado bajo un enfoque centrado en el usuario, considerando los tres perfiles principales: docente, estudiante y administrador. Cada rol posee necesidades específicas, por lo que la interfaz fue estructurada para facilitar la interacción según sus funciones dentro de la app.

- **Enfoque para el Docente**

  El docente es uno de los usuarios principales de la app, por lo que se priorizó:

  - Creación rápida y organizada de clases.
  - Publicación sencilla de materiales y tareas.
  - Visualización clara de estudiantes inscritos.
  - Confirmaciones antes de cancelar acciones importantes.
  - Mensajes de éxito tras completar procesos.

- **Enfoque para el Estudiante**

  Para el estudiante, se priorizó una experiencia simple y directa, enfocada en el acceso y consulta de información.

  Se consideró:

  - Proceso intuitivo para unirse a clases mediante código.
  - Visualización clara de materiales y tareas publicadas.
  - Navegación sencilla entre secciones.
  - Mensajes comprensibles en caso de error o falta de datos.
  - Interfaz limpia para evitar sobrecarga visual.

- **Enfoque para el Administrador**

  El administrador cumple una función de supervisión y control dentro del sistema. Por ello, el diseño contempla:

  - Gestión de usuarios (docentes y estudiantes).
  - Supervisión de clases creadas.

- Control de información general de la app.
- Acceso a reportes o estadísticas.

La interfaz del administrador se caracteriza por ser estructurada, organizada y enfocada en la gestión global de la app, manteniendo coherencia visual con los demás roles, pero con funcionalidades ampliadas.

- **Integración de los Tres Roles**

Aunque cada perfil tiene funciones distintas, la app mantiene una línea gráfica uniforme y una estructura coherente, permitiendo que la experiencia general sea consistente independientemente del rol que utilice la aplicación.

- Este enfoque garantiza:
- Claridad en las funciones de cada usuario.
- Reducción de errores.
- Mejor organización de la app

El diseño UX/UI desarrollado para la aplicación integra principios de usabilidad, coherencia visual y navegación intuitiva. Además, garantiza una experiencia organizada y profesional

4.Explicación de la lógica de solución

El proceso de autenticación en el Sistema Educational Control se inicia cuando un usuario accede a la aplicación e ingresa su correo electrónico y contraseña. Estos datos se transmiten al servidor, el cual procede a validar las credenciales mediante una consulta a la base de datos. El servidor localiza el registro del usuario correspondiente al correo proporcionado y verifica la coincidencia entre la contraseña ingresada y la almacenada de forma segura. En caso de validación exitosa, se determina el rol del usuario (estudiante, docente o administrador) y se recuperan los datos asociados desde las tablas relacionadas. Posteriormente, se obtienen las clases vinculadas al usuario, ya sea aquellas impartidas por el docente o en las que el estudiante está inscrito. Finalmente, el servidor responde con la información consolidada del usuario, permitiendo que la aplicación renderice el dashboard adaptado al rol correspondiente.

4.1 Pasos de la Lógica de Solución

1. **Interacción del Usuario en la Aplicación Móvil**

El usuario accede a la aplicación Educational Control y selecciona la opción de inicio de sesión, ingresando su correo electrónico y contraseña en los campos designados. La aplicación ejecuta una validación preliminar para confirmar el formato válido del correo y que la contraseña no esté vacía. Una vez cumplidos estos requisitos, se envía una solicitud HTTP al servidor con las credenciales.

2. **Recepción y Primera Validación en el Servidor**

El servidor recibe la solicitud y realiza una validación exhaustiva, verificando el formato del correo y la presencia de campos obligatorios. Si los datos no cumplen con los criterios establecidos, se retorna un mensaje de error descriptivo al cliente.

3. **Consulta del Usuario en la Base de Datos**

De ser exitosa la validación, el servidor consulta la tabla *Usuario* para identificar el registro asociado al correo electrónico proporcionado. En ausencia del usuario, se notifica un error de credenciales inválidas. Si el usuario existe, se compara la contraseña ingresada con la almacenada, empleando mecanismos de hash para garantizar la seguridad.

4. **Identificación del Rol del Usuario**

Tras confirmar la contraseña, el sistema extrae el campo *rol* del registro del usuario, clasificándolo como estudiante, docente o administrador. Esta identificación define las funcionalidades y permisos accesibles en el sistema

5. **Carga de Información Específica del Rol**

Conforme al rol detectado, se consultan las tablas adyacentes para enriquecer el perfil. Por ejemplo, para docentes se accede a la tabla *Docente* con sus datos asociados; para estudiantes, a la tabla *Estudiante* para

obtener el Número de Identificación Estudiantil (NIE) y la institución afiliada. Estos elementos se integran al perfil básico del usuario.

6. **Consulta de Clases Asociadas**

Se recuperan las clases relevantes: para docentes, aquellas en las que figuran como responsables en la tabla *Clase*; para estudiantes, mediante la tabla *Inscripción*, que vincula estudiantes con materias inscritas, generando así la lista de clases activas

7. **Generación de la Respuesta del Sistema**

El servidor compila una respuesta estructurada que incluye los datos básicos del usuario, su rol, el perfil específico y la lista de clases asociadas, la cual se envía de vuelta a la aplicación.

8. **Procesamiento en la Aplicación**

La aplicación almacena la información recibida en la sesión activa, carga las clases asociadas y configura la interfaz principal del sistema

9. **Navegación según el Rol**

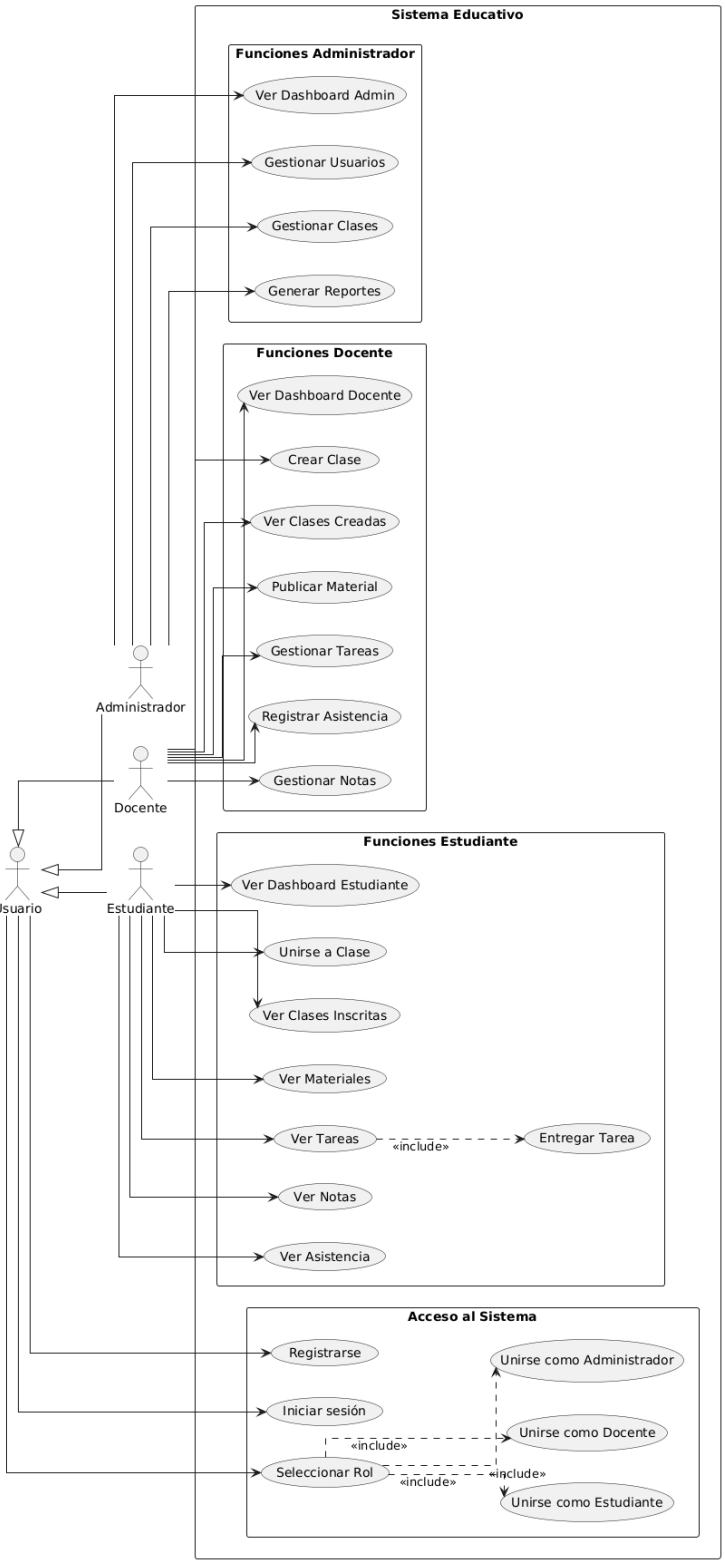
El usuario es redirigido al dashboard personalizado: los estudiantes visualizan clases, tareas, materiales, notas y asistencia; los docentes gestionan clases, publican materiales, registran asistencia y asignan tareas; los administradores acceden a herramientas de gestión de usuarios, clases y reportes.

4.2 Resumen de los pasos:

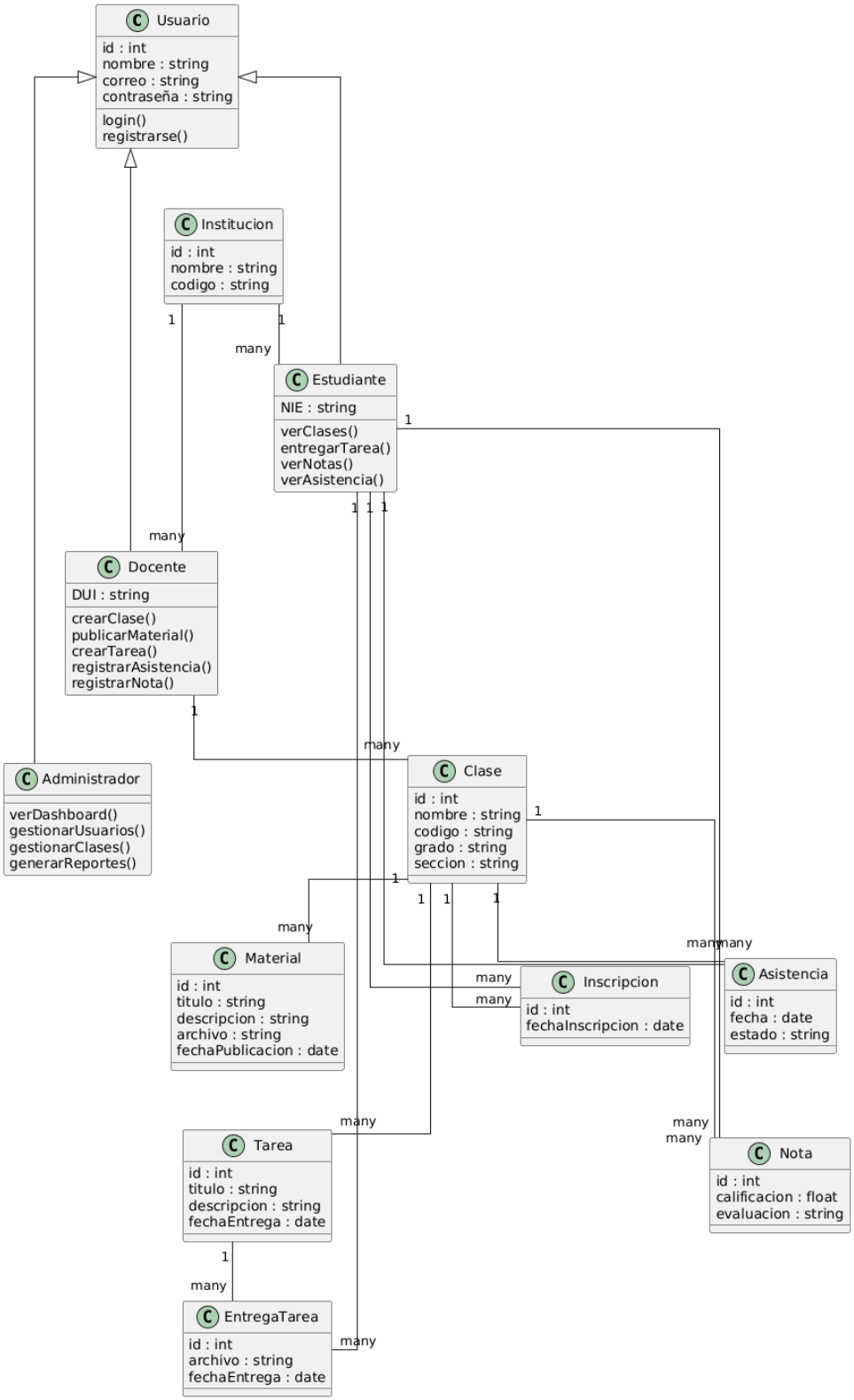
Paso	Componente	Acción Principal	Datos Involucrados	Resultado
1	Aplicación	Usuario ingresa correo y contraseña	Correo, contraseña	Validación local exitosa
2	Servidor	Validación de datos recibidos	Correo, contraseña	Datos válidos confirmados
3	Base de Datos	Consulta de usuario	Tabla *Usuario*	Usuario encontrado
4	Servidor	Identificación del rol	Campo *rol_usuario*	Rol determinado
5	Base de Datos	Consulta información del perfil	Tablas *Estudiante* / *Docente*	Perfil cargado
6	Base de Datos	Consulta clases asociadas	Tablas *Clase* / *Inscripción*	Lista de clases obtenida
7	Servidor	Preparación de respuesta	Datos usuario + clases	Respuesta enviada
8	Aplicación	Configuración de sesión	Perfil usuario	Sesión iniciada
9	Aplicación	Redirección según rol	Tipo de usuario	Dashboard correspondiente

4.3 Diagramas

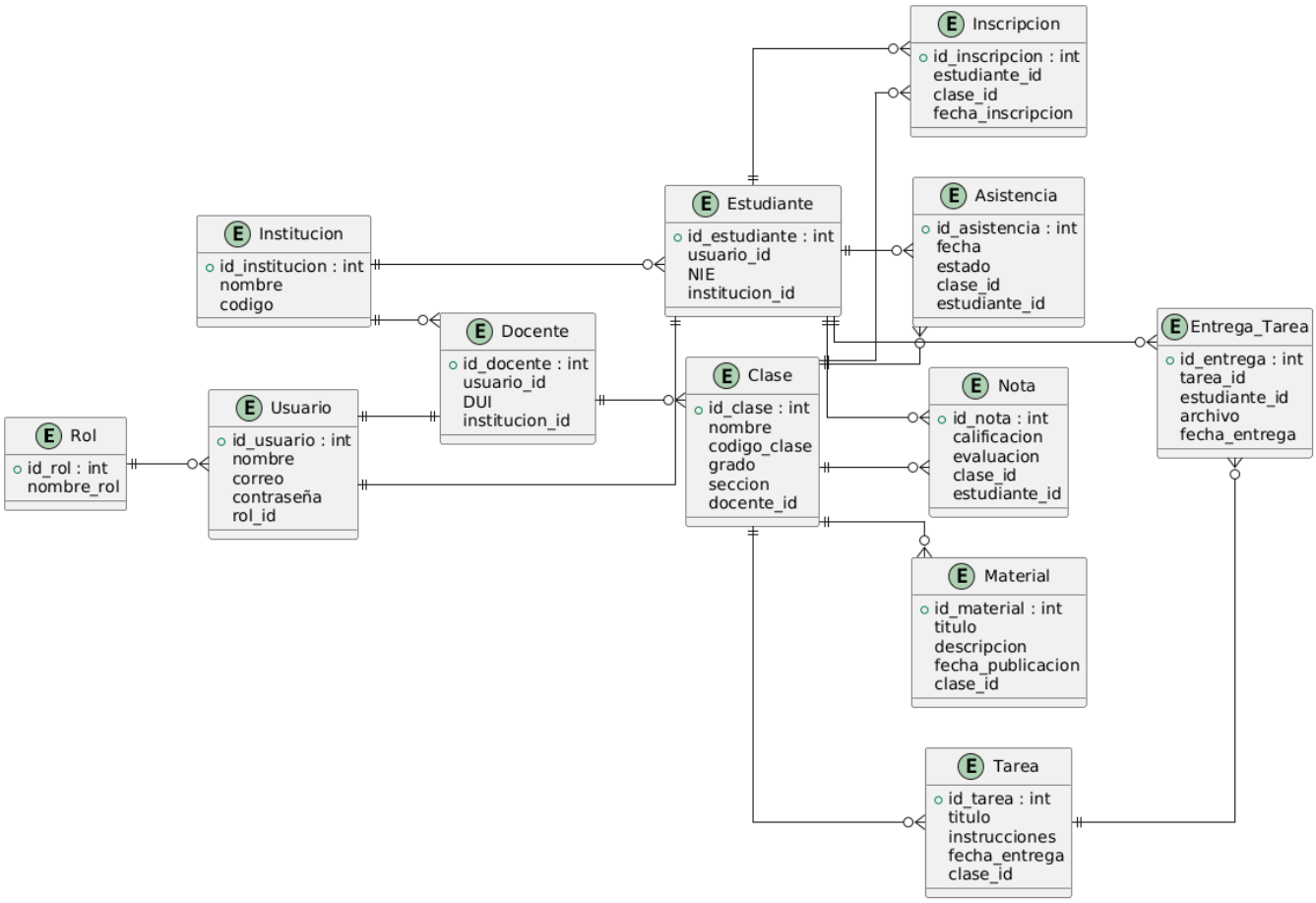
4.3.1 Diagrama de Caso de Uso



4.3.2 Diagrama de Clases

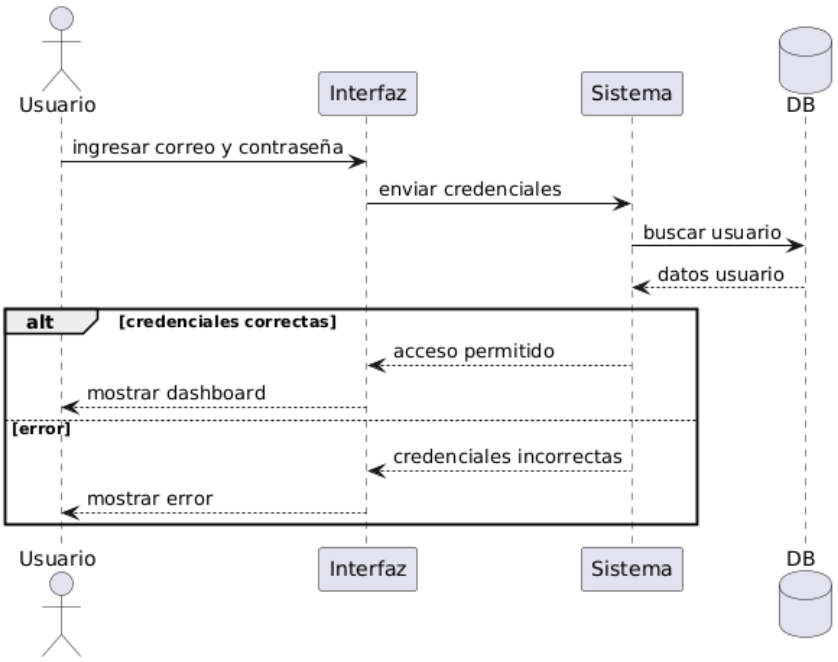


4.3.3 Diagrama de Entidad Relación

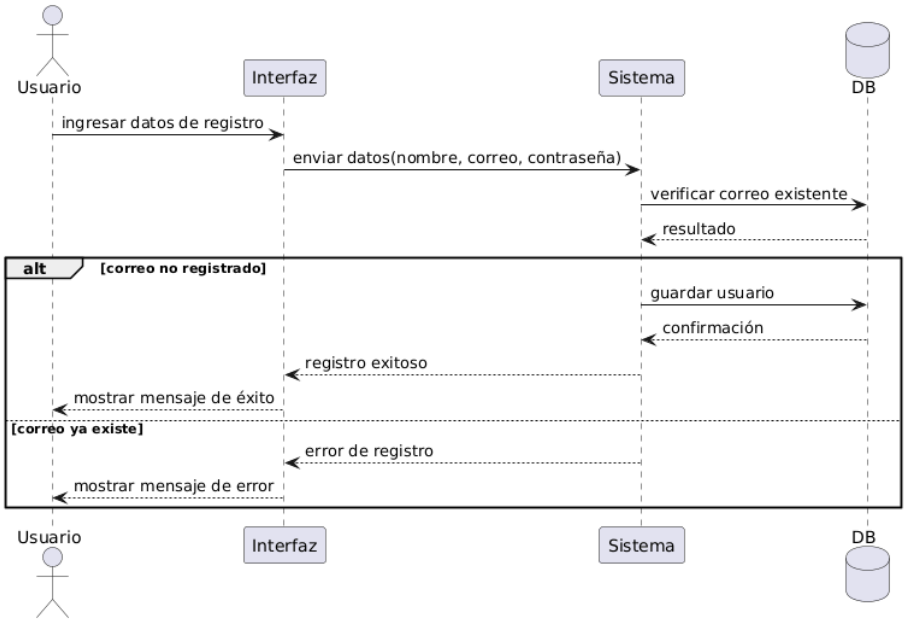


4.3.4 Diagrama de Secuencia

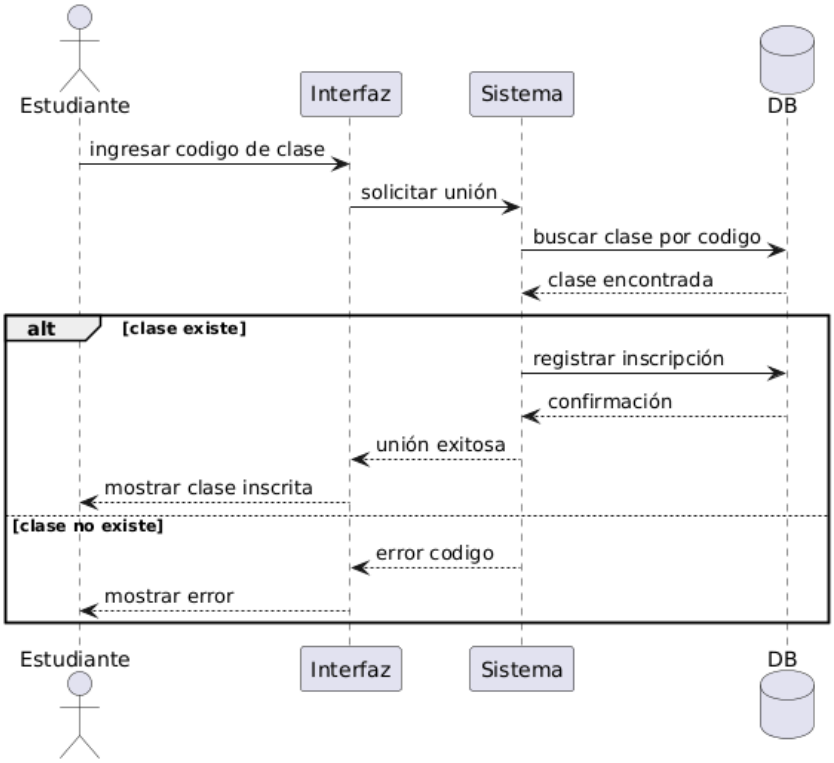
1. **Login**



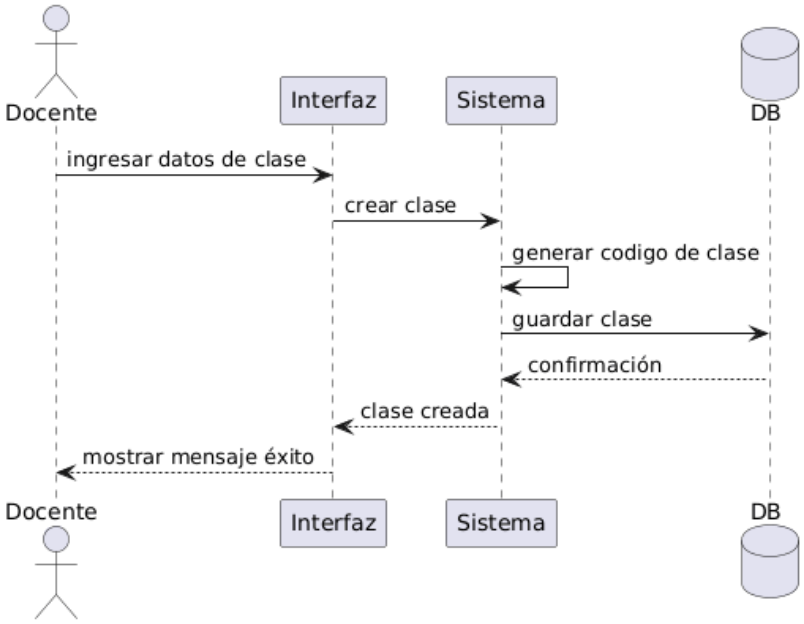
2. **Registro**



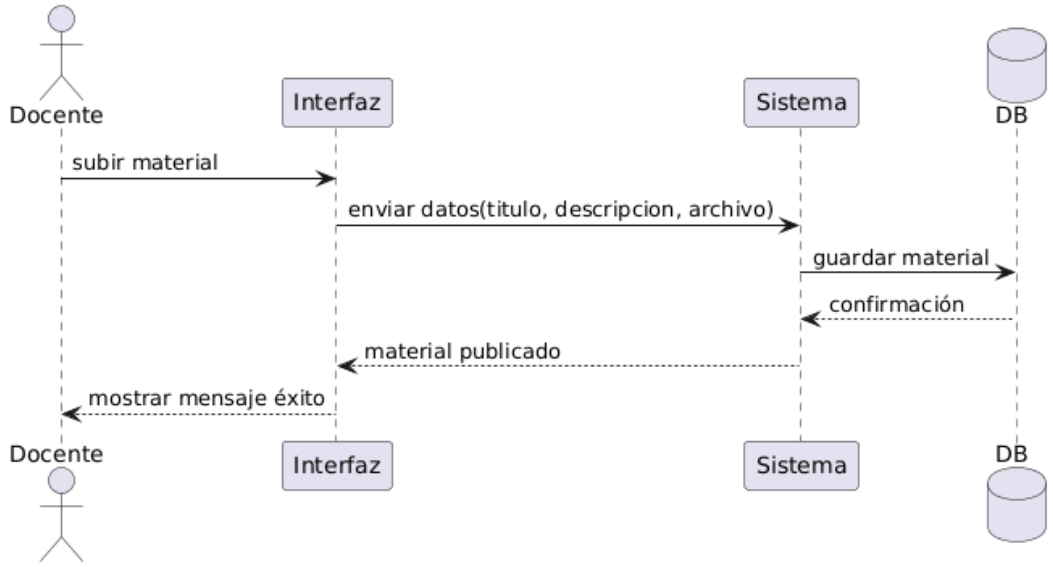
3. Unirse a Clase



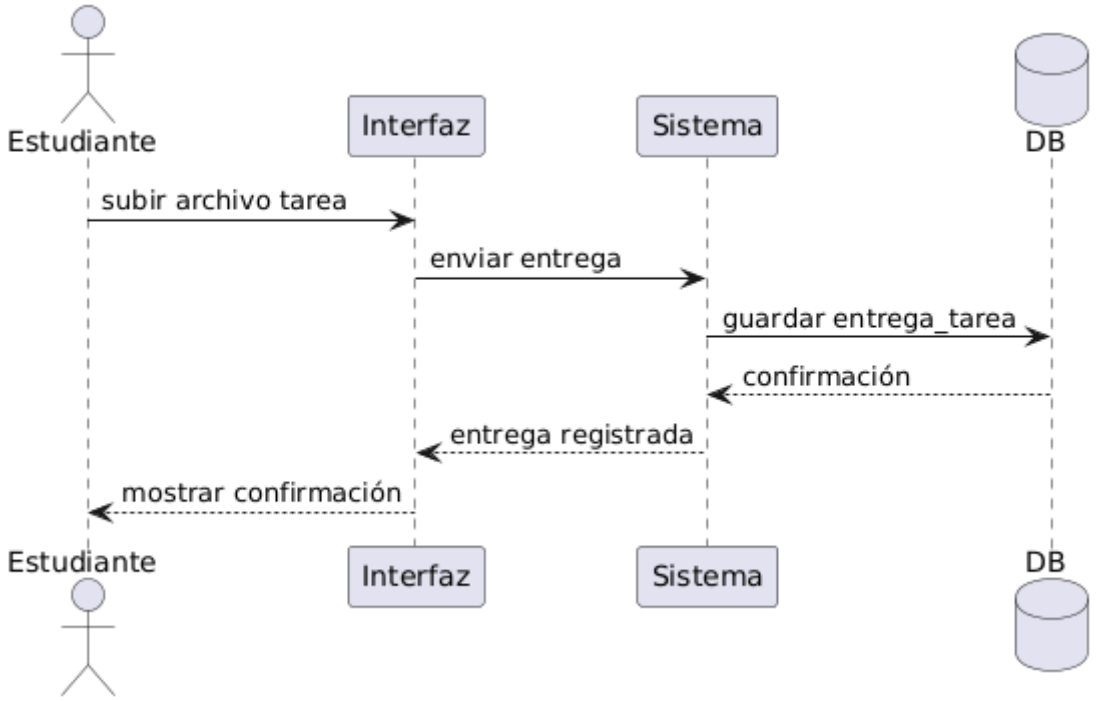
4. Crear Clase



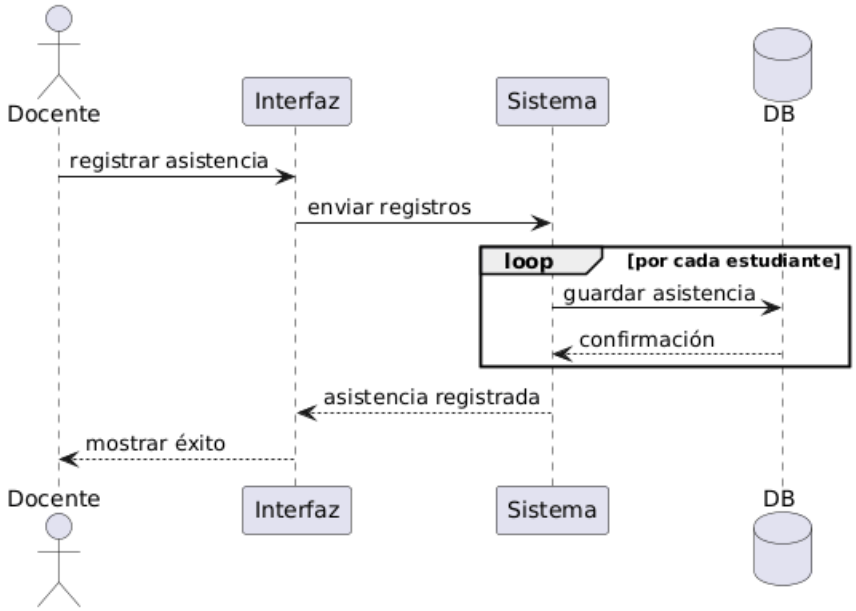
5. Publicar Material



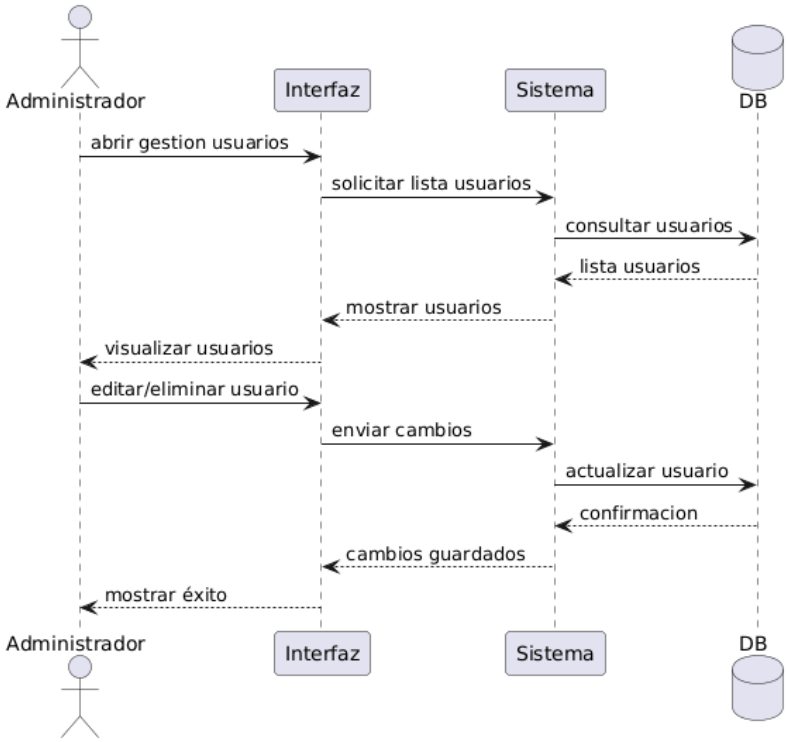
6. Entregar Tarea



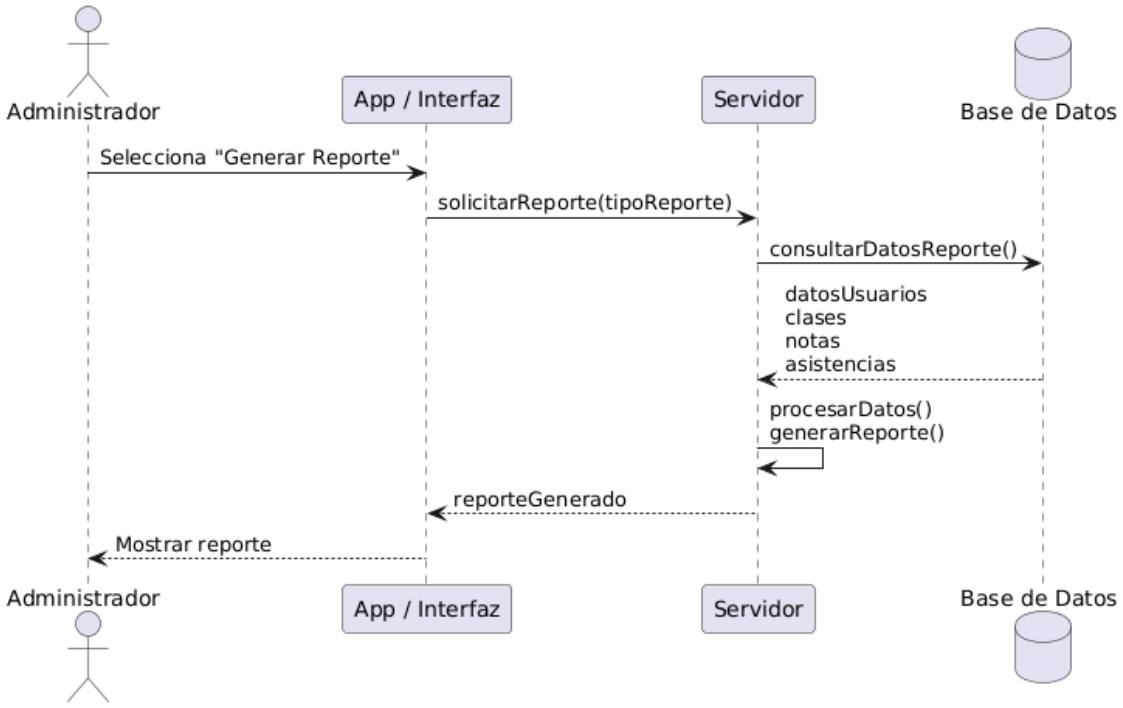
7. Registrar Asistencia



8. Gestionar Usuarios

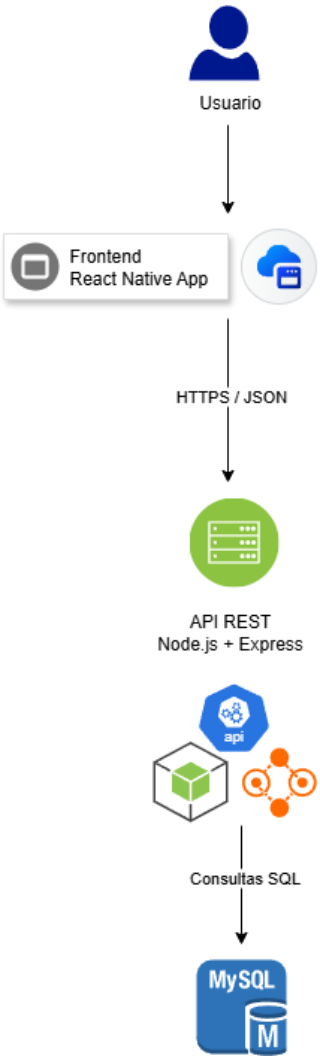


9. Generar Reporte



5.Arquitectura de la aplicación

Diagrama gráfico de la arquitectura de software.



La arquitectura del sistema Educational Control estará basada en un modelo cliente-servidor, el cual permite separar la interfaz de usuario de la lógica de negocio y del almacenamiento de datos.

Este enfoque facilita el mantenimiento del sistema, mejora la escalabilidad de la aplicación y permite que las diferentes partes del sistema funcionen de manera independiente.

La arquitectura estará compuesta por tres componentes principales: el cliente, el servidor y la base de datos.

Backend y lógica de negocio.

El backend será el encargado de procesar la lógica del sistema y gestionar la comunicación entre la aplicación móvil y la base de datos.

Para el desarrollo del backend se utilizará Node.js junto con el framework Express, el cual permitirá construir una API REST que gestionará las solicitudes enviadas desde la aplicación móvil.

El servidor será responsable de validar los datos enviados por los usuarios, ejecutar las operaciones necesarias en la base de datos y enviar respuestas a la aplicación.

Base de datos y modelo de datos.

DBMS seleccionado

El sistema **Educational Control** utilizará como gestor de base de datos **MySQL**.

Justificación

Se seleccionó MySQL por las siguientes razones:

- Es un sistema **relacional estable y ampliamente utilizado**.
- Tiene **alta compatibilidad con Node.js, Laravel y Python**, que son los frameworks sugeridos para el backend.
- Permite manejar **grandes volúmenes de datos académicos** de forma eficiente.
- Es **open source** y fácil de implementar en servidores cloud.

- Permite implementar **relaciones entre tablas, claves foráneas y consultas complejas** necesarias para reportes académicos.

Además, MySQL puede integrarse fácilmente con **APIs REST y sistemas web desarrollados en React o React Native**, lo cual es la arquitectura planteada en el proyecto.

Modelo de Datos

El sistema almacenará la información académica mediante un **modelo relacional** compuesto por las siguientes tablas principales:

- usuario
- clase
- estudiante_clase
- tarea
- asistencia
- nota

Este modelo permite gestionar usuarios, clases, inscripciones, tareas, asistencia y calificaciones.

Base de datos

```
CREATE DATABASE educational_control;
USE educational_control;

CREATE TABLE usuario (
   id CHAR(36) PRIMARY KEY,
   nombre VARCHAR(100) NOT NULL,
   email VARCHAR(100) NOT NULL UNIQUE,
   password_hash VARCHAR(255) NOT NULL,
   rol ENUM('docente', 'estudiante', 'admin') NOT NULL,
   fcm_token VARCHAR(255)
);

CREATE TABLE clase (
   id CHAR(36) PRIMARY KEY,
   nombre VARCHAR(100) NOT NULL,
   codigo_qr VARCHAR(100) NOT NULL UNIQUE,
   docente_id CHAR(36) NOT NULL,
   CONSTRAINT fk_clase_docente
      FOREIGN KEY (docente_id) REFERENCES usuario(id)
);

CREATE TABLE estudiante_clase (
   id CHAR(36) PRIMARY KEY,
   usuario_id CHAR(36) NOT NULL,
   clase_id CHAR(36) NOT NULL,
   CONSTRAINT fk_estudiante_clase_usuario
```

```
    FOREIGN KEY (usuario_id) REFERENCES usuario(id),
  CONSTRAINT fk_estudiante_clase_clase
    FOREIGN KEY (clase_id) REFERENCES clase(id),
  CONSTRAINT uq_estudiante_clase UNIQUE (usuario_id, clase_id)
);

CREATE TABLE tarea (
  id CHAR(36) PRIMARY KEY,
  titulo VARCHAR(150) NOT NULL,
  fecha_limite DATE NOT NULL,
  clase_id CHAR(36) NOT NULL,
  CONSTRAINT fk_tarea_clase
    FOREIGN KEY (clase_id) REFERENCES clase(id)
);

CREATE TABLE asistencia (
  id CHAR(36) PRIMARY KEY,
  estudiante_id CHAR(36) NOT NULL,
  clase_id CHAR(36) NOT NULL,
  fecha DATE NOT NULL,
  presente BOOLEAN NOT NULL,
  CONSTRAINT fk_asistencia_estudiante
    FOREIGN KEY (estudiante_id) REFERENCES usuario(id),
  CONSTRAINT fk_asistencia_clase
    FOREIGN KEY (clase_id) REFERENCES clase(id),
  CONSTRAINT uq_asistencia UNIQUE (estudiante_id, clase_id, fecha)
);
```

```
CREATE TABLE nota (
   id CHAR(36) PRIMARY KEY,
   valor FLOAT NOT NULL,
   estudiante_id CHAR(36) NOT NULL,
   clase_id CHAR(36) NOT NULL,
   CONSTRAINT fk_nota_estudiante
      FOREIGN KEY (estudiante_id) REFERENCES usuario(id),
   CONSTRAINT fk_nota_clase
      FOREIGN KEY (clase_id) REFERENCES clase(id)
);
```

Tablas de la base de datos:

Tabla: asistencia

Registra la asistencia del estudiante en cada clase.

Campos principales:

- **id**
- **estudiante_id**
- **clase_id**
- **fecha**
- **presente**

Función:

Permite controlar si un estudiante asistió o no a una clase en una fecha determinada.

asistencia

Columna	Tipo	Nulo	Predeterminado	Comentarios
id *(Primaria)*	char(36)	No		
estudiante_id	char(36)	No		
clase_id	char(36)	No		
fecha	date	No		
presente	tinyint(1)	No		

Índices

Nombre de la clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo	Comentario
PRIMARY	BTREE	Sí	No	id	0	A	No	
uq_asistencia	BTREE	Sí	No	estudiante_id	0	A	No	
				clase_id	0	A	No	
				fecha	0	A	No	
fk_asistencia_clase	BTREE	No	No	clase_id	0	A	No	

Tabla: clase

Registra las clases creadas en el sistema.

Campos principales:

- **id**
- **nombre**
- **codigo_qr**
- **docente_id**

Función:

Permite registrar cada clase y asociarla al docente que la imparte.

clase

Columna	Tipo	Nulo	Predeterminado	Comentarios
id *(Primaria)*	char(36)	No		
nombre	varchar(100)	No		
codigo_qr	varchar(100)	No		
docente_id	char(36)	No		

Índices

Nombre de la clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo	Comentario
PRIMARY	BTREE	Sí	No	id	0	A	No	
codigo_qr	BTREE	Sí	No	codigo_qr	0	A	No	
fk_clase_docente	BTREE	No	No	docente_id	0	A	No	

Tabla: estudiante_clase

Tabla intermedia que relaciona estudiantes con clases.

Campos principales:

- **id**
- **usuario_id**
- **clase_id**

Función:

Permite inscribir estudiantes en una o varias clases.

estudiante_clase

Columna	Tipo	Nulo	Predeterminado	Comentarios
id *(Primaria)*	char(36)	No		
usuario_id	char(36)	No		
clase_id	char(36)	No		

Índices

Nombre de la clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo	Comentario
PRIMARY	BTREE	Sí	No	id	0	A	No	
uq_estudiante_clase	BTREE	Sí	No	usuario_id	0	A	No	
				clase_id	0	A	No	
fk_estudiante_clase_clase	BTREE	No	No	clase_id	0	A	No	

Tabla: nota

Guarda las calificaciones de los estudiantes por clase.

Campos principales:

- **id**
- **valor**
- **estudiante_id**
- **clase_id**

Función:

Permite registrar la nota obtenida por un estudiante dentro de una clase.

nota

Columna	Tipo	Nulo	Predeterminado	Comentarios
id *(Primaria)*	char(36)	No		
valor	float	No		
estudiante_id	char(36)	No		
clase_id	char(36)	No		

Índices

Nombre de la clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo	Comentario
PRIMARY	BTREE	Sí	No	id	0	A	No	
fk_nota_estudiante	BTREE	No	No	estudiante_id	0	A	No	
fk_nota_clase	BTREE	No	No	clase_id	0	A	No	

Tabla: tarea

Guarda las tareas asignadas en cada clase.

Campos principales:

- **id**
- **titulo**
- **fecha_limite**
- **clase_id**

Función:

Permite registrar actividades o tareas pendientes asociadas a una clase.

tarea

Columna	Tipo	Nulo	Predeterminado	Comentarios
id *(Primaria)*	char(36)	No		
titulo	varchar(150)	No		
fecha_limite	date	No		
clase_id	char(36)	No		

Índices

Nombre de la clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo	Comentario
PRIMARY	BTREE	Sí	No	id	0	A	No	
fk_tarea_clase	BTREE	No	No	clase_id	0	A	No	

Tabla: usuario

Guarda la información general de las personas dentro del sistema.

Campos principales:

- **id**
- **nombre**
- **email**

- **password_hash**
- **rol**
- **fcm_token**

Función:

Permite autenticar usuarios y definir su tipo dentro del sistema, por ejemplo: docente o estudiante.

usuario

Columna	Tipo	Nulo	Predeterminado	Comentarios
id *(Primaria)*	char(36)	No		
nombre	varchar(100)	No		
email	varchar(100)	No		
password_hash	varchar(255)	No		
rol	enum('docente', 'estudiante', 'admin')	No		
fcm_token	varchar(255)	Sí	*NULL*	

Índices

Nombre de la clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo	Comentario
PRIMARY	BTREE	Sí	No	id	0	A	No	
email	BTREE	Sí	No	email	0	A	No	

Relaciones entre tablas

Usuario → Clase (1:N)

Un usuario con rol de docente puede impartir muchas clases.

- **clase.docente_id → usuario.id**

Usuario → Estudiante_Clase (1:N)

Un usuario con rol de estudiante puede estar inscrito en muchas clases.

- **estudiante_clase.usuario_id → usuario.id**

Clase → Estudiante_Clase (1:N)

Una clase puede tener muchos estudiantes inscritos.

- **estudiante_clase.clase_id → clase.id**

Clase → Tarea (1:N)

Una clase puede contener muchas tareas.

- **tarea.clase_id → clase.id**

Usuario → Asistencia (1:N)

Un estudiante puede tener múltiples registros de asistencia.

- **asistencia.estudiante_id → usuario.id**

Clase → Asistencia (1:N)

Una clase puede tener varios registros de asistencia.

- **asistencia.clase_id → clase.id**

Usuario → Nota (1:N)

Un estudiante puede tener varias notas.

- **nota.estudiante_id → usuario.id**

Clase → Nota (1:N)

Una clase puede tener muchas notas registradas.

- **nota.clase_id → clase.id**

Claves primarias y foráneas

Claves primarias (PK)

Cada tabla posee una clave primaria única:

- **usuario.id**
- **clase.id**

- **estudiante_clase.id**
- **tarea.id**
- **asistencia.id**
- **nota.id**

Claves foráneas (FK)

- **clase.docente_id → usuario.id**
- **estudiante_clase.usuario_id → usuario.id**
- **estudiante_clase.clase_id → clase.id**
- **tarea.clase_id → clase.id**
- **asistencia.estudiante_id → usuario.id**
- **asistencia.clase_id → clase.id**
- **nota.estudiante_id → usuario.id**
- **nota.clase_id → clase.id**

API y protocolos de comunicación.

La comunicación entre la aplicación móvil y el servidor se realizará mediante una API REST utilizando el protocolo HTTP/HTTPS.

Las solicitudes enviadas por la aplicación serán procesadas por el servidor, el cual responderá enviando datos en formato JSON.

Este mecanismo permite mantener una comunicación eficiente y estructurada entre los diferentes componentes del sistema.

Gestión de sesiones

El sistema utilizará tokens de autenticación para gestionar las sesiones de los usuarios. Cuando un usuario inicie sesión correctamente, el servidor generará un token de autenticación que permitirá identificar al usuario durante su interacción con el sistema.

Este token será enviado en cada solicitud posterior para verificar que el usuario tenga permisos para acceder a las funcionalidades del sistema.

Estrategia de seguridad

El sistema implementará medidas básicas de seguridad para proteger la información académica almacenada.

Entre las medidas de seguridad se incluyen:

- autenticación de usuarios mediante credenciales
- control de permisos según el tipo de usuario
- validación de datos ingresados en los formularios
- protección contra ataques comunes como inyección SQL

Estas medidas permiten garantizar la confidencialidad y seguridad de los datos.

6. Herramientas y tecnologías a utilizar

6.1. Lenguajes y frameworks

Para el desarrollo del sistema Educational Control se utilizarán tecnologías modernas que permitan construir una aplicación robusta y escalable.

El lenguaje principal del proyecto será **JavaScript**, el cual será utilizado tanto en el desarrollo del frontend como del backend del sistema.

Para el desarrollo de la aplicación móvil se utilizará **React Native**, framework que permite crear aplicaciones móviles multiplataforma utilizando una única base de código.

En el lado del servidor se utilizará **Node.js** junto con el framework **Express.js**, lo que permitirá desarrollar una API REST encargada de gestionar las solicitudes de la aplicación y la comunicación con la base de datos.

6.2. Gestor de base de datos

Para el almacenamiento de la información académica del sistema se utilizará **MySQL**, un sistema de gestión de bases de datos relacional ampliamente utilizado que permitirá administrar de forma segura los datos de estudiantes, calificaciones, asistencia y usuarios del sistema.

6.3. Control de versiones Git/GitHub

Para el control de versiones del proyecto se utilizará **Git**, que permitirá gestionar los cambios realizados en el código fuente durante el desarrollo.

Asimismo, se empleará la plataforma **GitHub**, la cual facilitará el trabajo colaborativo entre los integrantes del equipo mediante repositorios compartidos, control de versiones y seguimiento del progreso del proyecto.

6.4. Estrategia de CI/CD y herramientas seleccionadas

Para automatizar los procesos de integración y despliegue continuo se utilizará **GitHub Actions**, herramienta que permitirá ejecutar flujos de trabajo automatizados cada vez que se realicen cambios en el repositorio del proyecto.

Esto permitirá realizar pruebas automáticas, verificar la correcta integración del código y mantener una gestión más eficiente del desarrollo del sistema.

6.5. Herramientas complementarias

Además, se utilizarán herramientas de apoyo para el diseño y la gestión del proyecto:

- **Figma** para el diseño de los prototipos y mockups de la aplicación.
- **Trello** para la planificación y seguimiento de las tareas mediante tableros de trabajo basados en metodologías ágiles.

7. Licenciamiento del proyecto

7.1 Tipo de Licencia Creative Commons seleccionada

Para el presente proyecto se ha seleccionado la licencia **Creative Commons Atribución 4.0 Internacional (CC BY 4.0)**.

Esta licencia permite que el contenido del proyecto pueda ser compartido, copiado, distribuido y adaptado por terceros, incluso con fines comerciales, siempre que se otorgue el debido reconocimiento a los autores originales del trabajo.

La licencia Creative Commons BY 4.0 es ampliamente utilizada en proyectos académicos y de investigación, ya que promueve el acceso abierto al conocimiento, facilita la colaboración entre desarrolladores y permite que otros puedan estudiar, reutilizar o mejorar los recursos desarrollados.

Además, esta licencia favorece la difusión del proyecto Educational Control, permitiendo que pueda ser utilizado como referencia en futuros trabajos académicos, investigaciones o desarrollos tecnológicos relacionados con la gestión educativa.

7.2 Justificación de su elección y forma de implementación.

La elección de la licencia **Creative Commons Atribución 4.0 Internacional (CC BY 4.0)** se debe a que ofrece un equilibrio adecuado entre la **protección de la autoría del proyecto y la libertad de uso del contenido desarrollado**.

Al permitir que otras personas puedan utilizar, modificar y redistribuir el proyecto, se fomenta la **difusión del conocimiento y el aprendizaje colaborativo**, lo cual es especialmente importante en el ámbito académico y tecnológico.

Asimismo, esta licencia garantiza que los autores originales del proyecto reciban el **reconocimiento correspondiente por su trabajo**, ya que cualquier persona que utilice o modifique el proyecto deberá mencionar a los creadores originales.

La implementación de esta licencia dentro del proyecto se realizará mediante los siguientes elementos:

- Inclusión del **tipo de licencia Creative Commons seleccionada en el archivo README del repositorio de GitHub**.
- Incorporación de un **archivo LICENSE dentro del repositorio**, donde se especifican las condiciones de uso de la licencia CC BY 4.0.
- Inclusión de la **referencia a la licencia y su enlace oficial en la documentación del proyecto**.
- Identificación de los **autores del proyecto dentro del repositorio y del documento final**, con el fin de garantizar el reconocimiento correspondiente.

De esta manera, cualquier persona que tenga acceso al proyecto podrá conocer claramente las condiciones bajo las cuales el contenido puede ser utilizado, compartido o modificado.

8. Presupuesto del proyecto

Recurso	***Herramienta/Programa***	***Costo***	***Fases del proyecto***	***Horas estimadas***
Servidor backend (API)	Amazon Web Services / Microsoft Azure	$15 anual	Fase 2 – Implementación	40 horas
Base de datos	MySQL Cloud (RDS / Azure SQL)	$10 mensual	Fase 2 – Implementación	25 horas
Almacenamiento archivos	Cloud Storage (Google/AWS)	$5 mensual	Fase 2 – Implementación	15 horas
Certificado SSL	Let's Encrypt	Gratis	Fase 2 – Implementación	5 horas
Control de versiones	GitHub	Gratis	Fase 1 – Implementación	20 horas
Desarrollo backend	Node.js / Express	Gratis	Fase 2 – Implementación	50 horas
Desarrollo frontend	React / React Native	Gratis	Fase 2 – Implementación	30 horas
Diseño prototipos	Figma	Gratis	Fase 1 – Implementación	10 horas
Comunicación equipo	Discord / Teams	Gratis	Fase 1 – Implementación	15 horas
Mantenimiento servidor	Administrador técnico	$25 mensual	Fase 2 – Implementación	15 horas
Energía e internet	Operativo	$25 mensual	Fase 2 – Implementación	20 horas
Capacitación al personal	Manuales + inducción	$50 único pago	Fase 2 – Implementación	8 horas
Soporte técnico inicial	1 mes	$40	Fase 2 – Implementación	12 horas

Resumen del presupuesto

- Costo mensual estimado: **$80**
- Costos únicos iniciales: **$90**
- Costo aproximado del primer mes: **$170**
- Costo estimado del primer año: **$1050**

9. Cronograma de trabajo

Fase	Actividad	2-7 febrero	9-14 febrero	9-14 marzo	16-21 marzo	23-31 marzo	6-11 abril	13-18 abril
Perfil de proyecto	Definición de la idea del proyecto							
	Investigación del problema							
	Redacción de objetivos y metas							
	Elaboración del marco del proyecto							
	Elaboración del presupuesto							
	Revisión y corrección del documento							
	Entrega del perfil del proyecto							
Fase 1	Metodología de trabajo del equipo							
	Licenciamiento del proyecto							
	Diseño UX/UI y línea gráfica							
	Diseño del modelo UML inicial							

	Diseño de arquitectura del sistema							
	Diseño de base de datos							
Fase 2	Desarrollo del módulo de usuarios							
	Desarrollo del módulo de estudiantes							
	Desarrollo del módulo de asistencia							
	Desarrollo del módulo de calificaciones							
	Implementación de reportes académicos							
	Pruebas del sistema							
	Correcciones y ajustes finales							
	Presentación del sistema							

.

Enlaces

Link del prototipo: https://app.visily.ai/projects/1f8f37fe-c3c6-4700-bb67-c6bd4d414372/boards/2529238

Prototipo ya ejecutado: https://app.visily.ai/projects/1f8f37fe-c3c6-4700-bb67-c6bd4d414372/boards/2529238/presenter?play-mode=Prototype&flowId=f4d121d6-0dae-494a-8778-1927c13247f4

Link GitHub: https://github.com/kvrenderos/Educacional_Control_DPS

Bibliografias

Figma learn - help center. (n.d.). Figma.com. Retrieved March 16, 2026, from https://help.figma.com/

GitHub docs. (n.d.).

Introduction. (n.d.). Reactnative.dev. Retrieved March 16, 2026, from

https://reactnative.dev/docs/getting-started

Licenses list - creative commons. (n.d.). Creativecommons.org. Retrieved March 16, 2026, from

https://creativecommons.org/licenses/

Node.js v25.8.1 Documentation. (n.d.). Nodejs.org. Retrieved March 16, 2026, from

https://nodejs.org/en/docs

Scrum guides. (n.d.). Scrumguides.org. Retrieved March 16, 2026, from https://scrumguides.org/